



Tarefa Prática Mentoria Aula 03

Evoluindo o sistema para múltiplas wallets e estado interpretado

Objetivo

Na Aula 3, você construiu um sistema capaz de:

| criar, assinar, transmitir e acompanhar transações Bitcoin.

Agora, sua tarefa é evoluir esse sistema para um cenário mais próximo do mundo real:

| um backend que sabe operar com múltiplas wallets e interpretar o estado das transações acompanhadas.

A ideia não é criar um produto completo ainda.

Isso fica para o hackathon.

Aqui, o objetivo é adicionar **camadas de robustez e interpretação** sobre a aplicação já desenvolvida.

Contexto

Até agora, seu sistema já consegue:

- criar transações
- assinar com a wallet do Bitcoin Core
- transmitir via `sendrawtransaction`
- acompanhar o status `broadcast → mempool → confirmed`
- exibir transações no frontend

Mas sistemas reais raramente operam com apenas uma wallet ou apenas mostram estados crus.

Eles precisam responder perguntas como:

- Qual wallet estou usando?
- Essa transação está demorando demais?
- Ela já chegou na mempool?
- Ela confirmou recentemente?
- Posso trocar o contexto de execução sem alterar o código?

Esta tarefa representa esse próximo passo.

O que você deve desenvolver

1 Suporte a múltiplas wallets

Evolua seu backend para trabalhar explicitamente com wallets do Bitcoin Core.

Hoje, muitas chamadas RPC dependem de uma wallet específica, como:

- `listunspent`
- `getrawchangeaddress`
- `signrawtransactionwithwallet`
- `gettransaction`

Já outras chamadas são globais do node, como:

- `sendrawtransaction`
- `getblockchaininfo`
- `getmempoolentry`

Sua tarefa é separar esses dois contextos:

```
RPC do node:  
http://127.0.0.1:58443
```

```
RPC da wallet:  
http://127.0.0.1:58443/wallet/NOME_DA_WALLET
```

Você deve implementar:

- listagem de wallets existentes no node
- carregamento automático de wallet, se necessário
- seleção de wallet ativa
- uso da wallet selecionada para criar, assinar e acompanhar transações

Endpoints sugeridos

GET /wallets

Deve retornar:

```
{  
  "available_wallets": ["wallet1","wallet2"],  
  "loaded_wallets": ["wallet1"],  
  "selected_wallet": "wallet1"  
}
```

Use:

```
listwalletdir
```

para listar wallets existentes, e:

```
listwallets
```

para listar wallets carregadas.

POST /wallet/select

Body:

```
{  
  "wallet": "wallet2"
```

```
}
```

Deve:

- verificar se a wallet existe
- carregar com `loadwallet`, se ainda não estiver carregada
- definir a wallet ativa
- retornar informações básicas da wallet

Exemplo:

```
{
  "selected_wallet": "wallet2",
  "wallet_info": {
    "walletname": "wallet2",
    "balance": 0.001,
    "txcount": 4
  }
}
```

2 Campo de seleção de wallet no frontend

Atualize o frontend para exibir um campo de seleção de wallet.

Comportamento esperado:

- se existir apenas uma wallet, ela aparece selecionada automaticamente
- se existirem várias wallets, o usuário pode escolher em um `select`
- ao trocar a wallet, o backend deve atualizar a wallet ativa
- o envio de transação deve usar a wallet selecionada

A interface deve deixar claro:

| a transação será criada e assinada no contexto da wallet selecionada.

Na lista de transações enviadas, cada item deve mostrar também:

```
wallet: nome_da_wallet
```

3 Interpretação do estado da transação

Além de mostrar o status cru da transação, o sistema deve interpretar o que está acontecendo.

Você deve criar uma camada simples de interpretação para cada transação acompanhada.

Exemplos de interpretação:

Transação recém enviada

```
{
  "status": "broadcast",
  "message": "Transação enviada ao node, aguardando aceitação na mempool."
}
```

Transação na mempool

```
{
  "status": "mempool",
  "message": "Transação aceita na mempool, aguardando inclusão em bloco."
}
```

Transação demorando

```
{
  "status": "mempool",
  "warning": "Transação está na mempool há mais de 2 minutos."
}
```

Transação confirmada

```
{
  "status": "confirmed",
  "message": "Transação confirmada em bloco."
}
```

Transação não localizada

```
{
  "status": "unknown",
  "warning": "Transação não localizada na wallet selecionada."
}
```

4 Endpoint `/tx/<txid>` enriquecido

Atualize o endpoint de consulta de transação para retornar não apenas dados técnicos, mas também interpretação.

Deve retornar pelo menos:

- `txid`
- `wallet`
- `status`
- `confirmed`
- `confirmations`
- `block_hash`
- `age_seconds`
- `message`
- `warning`, se houver

Exemplo:

```
{
  "txid": "...",
  "wallet": "wallet1",
  "status": "mempool",
  "confirmed": false,
  "confirmations": 0,
  "block_hash": null,
  "age_seconds": 145,
  "message": "Transação aceita na mempool, aguardando inclusão em bloco.",
  "warning": "Transação está na mempool há mais de 2 minutos."
}
```

5 Saldo e UTXOs da wallet selecionada

mostrar informações básicas da wallet selecionada antes de enviar transações.

Crie um endpoint:

```
GET /wallet/status
```

Ele deve retornar:

```
{
  "wallet": "wallet1",
  "balance": 0.0012,
  "utxos": 3
}
```

Use:

- `getwalletinfo`
- `listunspent`

No frontend, adicione um pequeno card ou indicador mostrando:

- wallet selecionada
- saldo
- número de UTXOs disponíveis

Isso ajuda a conectar:

wallet → UTXOs → criação da transação.

Execução do sistema

O sistema deve estar acessível externamente, rodando em uma VPS ou através de um túnel (ex.: ngrok ou Cloudflare Tunnel), permitindo acesso ao frontend e à API a partir de outro dispositivo.